

Föreläsning – 260625 – ASP.NET CORE WEB.API – API Versioning

Tydlighet i förväntningar på dig som kursdeltagare?

→ Det förväntas att du implementerar dessa tips på ert Movie.API

när ni har tid.

→ För att ni ska kunna visa upp några projekt för en framtida arbetsgivare kommer du att ha dels Movie.API, dels slutprojektet.

Hur implementerar man versionshantering i sitt WEB.API?

API-versionering handlar om att kunna vidareutveckla ett API utan att bryta befintliga klienter — och det är betydligt svårare än vanlig versionshantering i .NET.

Varför API-versionering är svårare

I .NET (t.ex. assembly, kompilerad .NET-enhet, dll eller .exe):

- Konsumenten väljer själv vilken version som används.
- Flera versioner av en assembly kan finnas parallellt eller bytas ut mer kontrollerat, eftersom assemblyn är en tydlig paketerad komponent med egen identitet och version.

I ett API (ett publikt gränssnitt som externa klienter använder):

- Klienter är redan ute i produktion och kan inte tvingas uppdatera.
- Du måste stödja gamla och nya versioner samtidigt.
- Du har oftast en och samma deployment.
- Ändringar påverkar **direkt** riktiga användare.

Exempel: Byter du från firstName till givenName i JSON så slutar äldre klienter fungera.

Det är därför API-versionering handlar mer om **bakåtkompatibilitet och kontrakt** än teknik.

Vanliga strategier

1. URL-versionering (vanligast)

/api/v1/products

/api/v2/products

- Tydlig och enkel.
- Bra stöd i routing och Swagger.

2. Query string

/api/products?version=1

- Enkel att implementera.
- Mindre tydlig.

3. Header-versionering

GET /api/products

api-version: 1

- Renare URL.
- Svårare att testa och mindre synlig.

4. Media type (Accept header)

Accept: application/json;v=1

- Mest "REST-korrekt".
- Ovanlig i praktiken p.g.a. komplexitet.

Exempel i ASP.NET Core

```
builder.Services.AddApiVersioning(options =>
{
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.ReportApiVersions = true;
});
```

Controller:

```
[ApiController]
[Route ("api/v{version:apiVersion}/products")]
[ApiVersion("1.0")]    // talar om vilka versioner kontrollern deklarerar.
[ApiVersion("2.0")]
public class ProductsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetV1() => Ok("V1");

    [HttpGet]
    [MapToApiVersion("2.0")]    // talar om att just den här metoden ska
                                // användas när klienten anropar version 2.0
    public IActionResult GetV2() => Ok("V2");
}
```

[ApiVersion] = vilka versioner kontrollern har.

[MapToApiVersion] = vilken action som hör till vilken version.

Kontroller med endast en liten ändring i t.ex. en Product

```
using Asp.Versioning;
using Microsoft.AspNetCore.Mvc;

namespace MyApi.Controllers;

[ApiController]

[Route("api/v{version:apiVersion}/products")]

[ApiVersion("1.0")]
[ApiVersion("2.0")]

public class ProductsController : ControllerBase
{
    [HttpGet]
    [MapToApiVersion("1.0")]
    public IActionResult GetV1()
    {
        return Ok(new
        {
            Id = 1,
            Name = "Keyboard"
        });
    }

    [HttpGet]
    [MapToApiVersion("2.0")]
    public IActionResult GetV2()
    {
        return Ok(new
        {
            Id = 1,
            Name = "Keyboard",
            Price = 499,
            Currency = "SEK"
        });
    }
}
```

Vad som händer

När en request kommer in läser API-versioneringssystemet versionen från routen, eftersom vi använder `v{version:apiVersion}` i URL:en.

Därefter matchar systemet den versionen mot rätt action via `[MapToApiVersion]`.

Exempel på requests

Request till v1

`GET /api/v1/products`

Då väljs `GetV1()` eftersom den är mappad till "1.0".

Request till v2

`GET /api/v2/products`

Då väljs `GetV2()` eftersom den är mappad till "2.0".

Varför båda action-metoderna kan ligga i samma controller

Controllern deklarerar att den stödjer båda versionerna, men varje action talar om exakt vilken version den tillhör. Det är praktiskt när skillnaden är liten och man vill undvika att duplicera hela kontrollern.

Man kan ju också använda separata controllers

```
Controllers/  
├─ V1/  
│   └─ ProductsController.cs  
└─ V2/  
    └─ ProductsController.cs
```

Det här upplägget gör att varje version blir helt separat

namespace MyApi.Controllers.**V1**;

```
[ApiController] [Route("api/v{version:apiVersion}/products")]  
[ApiVersion("1.0")]  
public class ProductsController : ControllerBase  
{ }
```

namespace MyApi.Controllers.**V2**;

```
[ApiController] [Route("api/v{version:apiVersion}/products")]  
[ApiVersion("2.0")]  
public class ProductsController : ControllerBase { }
```

Program.cs

Se separat fil.

För att versioning ska fungera behöver man installera följande Nuget-paket:

För controller-baserad Web API-versionering behöver man:

- `Asp.Versioning.Mvc`
- `Asp.Versioning.ApiExplorer` om du vill ha Swagger-stöd för versioner.

```
dotnet add package Asp.Versioning.Mvc
```

```
dotnet add package Asp.Versioning.Mvc.ApiExplorer
```

Se klassens github-repo:

<https://github.com/Lexicon-LTU-VT-2026/WebAPI-Versioning>